Project Search Results Final

For the final search results, there really wasn't a whole lot that I could think of to still do. There really wasn't anything for me to change within the agents' evaluation functions, that was all sorted out in the initial results. However, I figured the best thing for me to do now was to run my agents in a greater number of games, as I only running 10 games per agent. I decided to run 25 games in each of the tests to get more accurate results. Here is a table representing the various outcomes I analyzed:

Results from 25 Games Per Pairing:

Defender Agent	Attacker Agent	Defender Win %	Attacker Win %	Average Length
Random	Random	76%	24%	115 Moves
Version 1	Random	100%	0%	20 Moves
Version 1	Version 1	20%	80%	35 Moves
Version 2	Version 1	32%	68%	44 Moves
Version 3	Version 1	80%	20%	28 Moves

So there are a few interesting takeaways from these results. For one, when both sides are selecting actions at random, the game is pretty heavily in favor of the defender. This is because the requirements for an attacker to win are a bit more complicated, and a lot more precise. In order for the attacker to win, they need to have 2 of their pieces on opposite sides, adjacent to the king to capture it and win. On the contrary, all the opposing team needs to do is get the king to any of the tiles on the 4 edges. This makes it much more likely for the defending team to win when making moves at random against a random opponent.

This imbalance is far more clear when the defender agent — having its basic search algorithm (minimax with alpha beta pruning) implemented — has a basic, but clear strategy for winning the game, which is simply to reach one of the edges. The average length of the game goes down from 115 moves to 20 on average per game. It's extremely unlikely for the attacker agent to be able to win in these conditions, and the data shows that in 25 games the attacker lost all of them.

Things start getting interesting in the third matches. Here it has shifted in favor of the attackers, due to the attacker's implementation of the evaluation function covering a lot more than the defender's does. This version of the attacker likes to play very aggressively. It evaluates whether moving a piece to a certain position will allow it to capture an enemy, or even if there's a position where an attacker can be moved to attack an opponent on the next turn. It favors being in positions where it's able to attack the king, and it's also aware of moving into spaces that will put its pieces in danger. The defender only ended up winning 20% of the 25 matches. The average length of the matches increased slightly, as to be expected when making both sides more intelligent.

Using the third matches as a brand new baseline, this is where I had decided that I was going to put all my attention on making the defender as strong as possible to try and beat the attacker, which was now a strong contender in its own right. The approach for version 2 was to update the evaluation function to now be aware of moving its pieces into positions that put it in danger. It does its best to focus on actually protecting the king, even sacrificing some of its pieces if deemed necessary. However, while this strategy proved to be an improvement upon what it was previously doing, it was still no match for the updated attacker agent.

While manually reviewing the matches, I realized that in almost every game, the defenders were getting almost entirely wiped out while the attackers kept the numbers. I did some research online for defending strategies, and I discovered a strategy that mainly involves the defenders aggressively taking out the attackers before letting the king escape to the edges.

The strategy involved a list of priorities for each turn. Never move the king, if you're able to capture a pawn then do so, if a pawn is out of its starting position and there's nowhere it can take an enemy pawn move it back into its original spot, and if all of your pieces are in the starting position and you can't immediately take a pawn, take the safest move possibly to as to drag the time out and bait your opponent into moving into a position where you can attack it.

Having implemented this strategy into the third version of the defender agent, it ended up working very effectively. Finally, the defense had won 80% of the 25 games. And not only that, but the number of moves per match had been cut nearly in half. With these results, I was happy with the current version of the defending agent. Establishing a new baseline where the attacker has a clear strategy, rather than relying on a random agent or something, definitely proved beneficial in the end.